

期末考试：

ppt所有内容弄懂；

课后题为考试真题

分治+dp+贪心.....80分

网络流、搜索、np概念（不考证明）20分

最后一道是开放题，一般是np完全问题（15分，没有快速方法，需要想其他的方法，近似、.....），暴力枚举9-10分，贪心解多1、2分

3周 周三-4周 周三：动态规划，5节课

4周 周三-4周 周五：贪心，3节课

5周 周三：摊还分析 1节

5周 周三-6周 周三：随机4节

6周 周三：NP 3节

 算法设计与分析_02_数学基础.pptx	2026/3/7 12:19
 算法设计与分析_03_分治.pptx	2026/3/23 23:01
 算法设计与分析_04_动态规划.pptx	2026/4/14 2:28
 算法设计与分析_05_贪心.pptx	2026/3/7 12:19
 算法设计与分析_06_摊还分析.pptx	2026/3/7 12:19
 算法设计与分析_07_随机.pptx	2026/3/7 12:19
 算法设计与分析_08_NP完全.pptx	2026/3/7 12:19
 算法设计与分析_09_近似.pptx	2026/3/7 12:19
 算法设计与分析_10_网络流.pptx	2026/4/23 12:28
 算法设计与分析_11_在线.pptx	2026/3/7 12:19
 算法设计与分析_12_搜索.pptx	2026/3/7 12:19

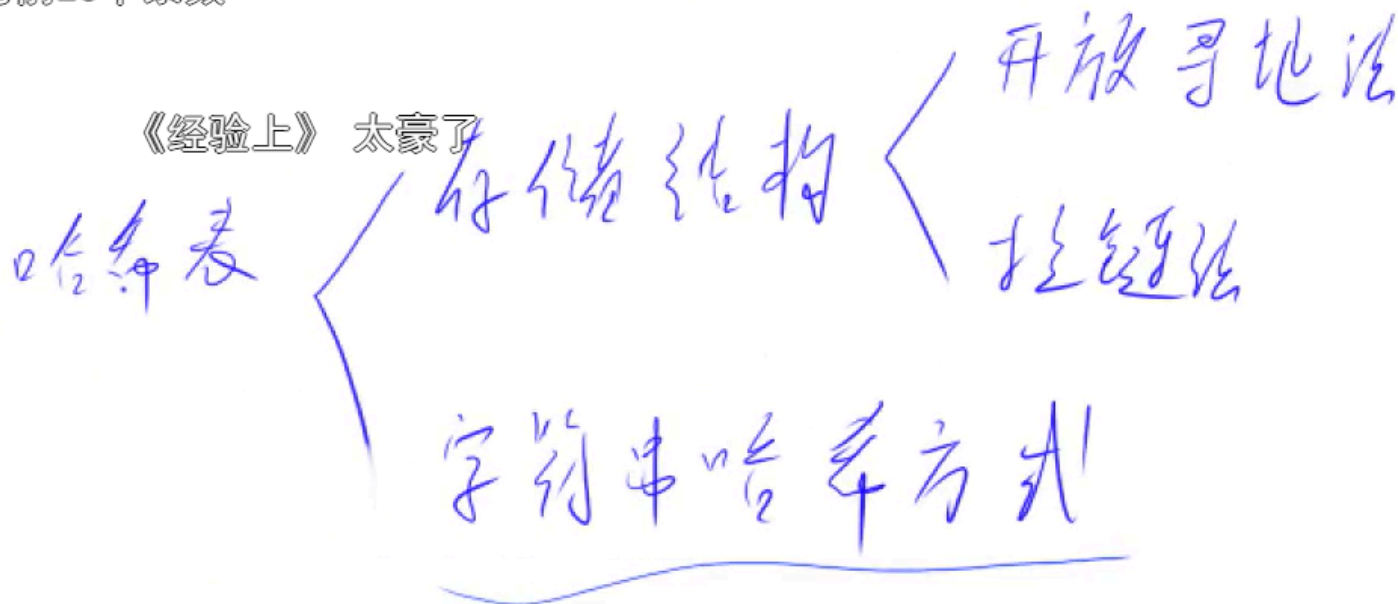
PPT 作业
✓

算法的本质

一、基本数据结构

1、哈希表：

《为刷LeetCode系统》



如果要快速判断两个字符串是否完全相同，可直接采用字符串哈希!!! $O(1)$!

2、并查集：路径压缩优化 $O(1)$

```
int find(int x) //“路径压缩”很重要!!! 不然会因效率低下而过不了!!
{
    if(p[x] != x) p[x] = find(p[x]);
    //1、注意此处递归必须传入p[x], 传入x会导致死循环
    //2、必须更新x的祖宗p[x]的值, 不能是x=find(), 意思都不对
    //3、这个递归很妙, 只有当x是根节点是才会执行到当前层的return返回自己的祖宗编号
    return p[x];
    //return x; 语义也不对, 应该是返回x的祖宗节点, 这样会导致无线套娃, Memory Limit Exceeded
}
///////////////////////////////////////////////////唯一写法//////////////////////////////////////
int find(int x)
{
    //简记: 函数体内全部用p[x]!!! (只有判断条件时会出现p[x] != x)
    if(p[x] != x) p[x] = find(p[x]);
    return p[x];
}
```

3、手写堆

如何手写一个堆？

1. 插入一个数
2. 求集合当中的最小值
3. 删除最小值
4. 删除任意一个元素
5. 修改任意一个元素

```
heap[ ++ size] = x; up(size);  
heap[1];  
heap[1] = heap[size]; size -- ; down(1);  
heap[k] = heap[size]; size -- ; down(k); up(k);  
heap[k] = x; down(k); up(k);
```

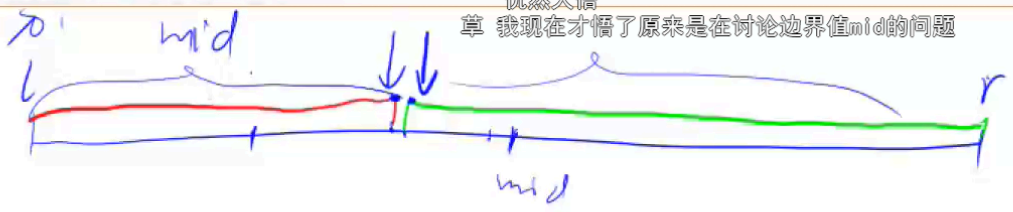
二、基本算法：

1、二分法

1.1（最终边界）区间收缩得到答案，while(l<r)

左右边界判定，可以找到距离答案最近的点（二分的是某种性质）

恍然大悟
草 我现在才悟了原来是在讨论边界值mid的问题



① $mid = \frac{l+r+1}{2}$

if (check(mid)) $\begin{cases} \text{true } [mid, r]: l=mid; \\ \text{false } [l, mid-1]: r=mid-1 \end{cases}$

② $mid = \frac{l+r}{2}$

if (check(mid)) $\begin{cases} \text{true } [l, mid]: r=mid \\ \text{false } [mid+1, r]: \end{cases}$

1.2 (搜索路径) 模拟一棵二分搜索树里的走路过程while(l<=r)

记录父亲的二分搜索：

父亲：这次查找路径上，走到目标前的上一个被比较的数。

```
int l=0,r=n-1;
int fa=-1;
int mid;
while(l<=r)
{
    mid=l+r>>1;

    if(a[mid]>x)r=mid-1;
    else if(a[mid]<x)l=mid+1;
    else break;

    fa=mid;
}
//默认题意是不会出现第一次命中的情况，不然fa=-1访问越界
if(a[mid]==x)cout<<"success, father is "<<a[fa]
else cout<<"not found, father is "<<a[fa]<<endl;
```


一、分治

(1)、快速排序: $O(n \log n)$

快速排序

分治



① 确定分界点: $q[l]$, $q[(l+r)/2]$, $q[r]$ 随机

② 调整区间: $\leq x$ $\geq x$ ☆

③ 递归处理左右两段

(2)、归并排序: $O(n \log n)$

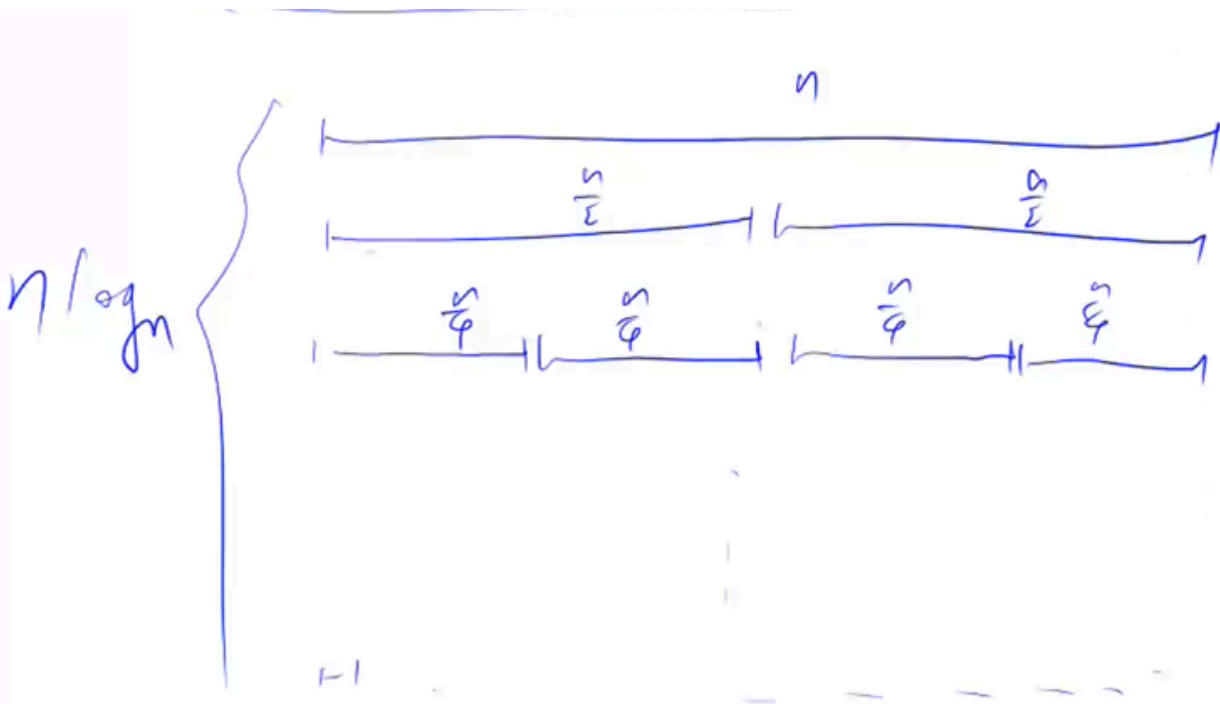
算法2: 归并排序 ——— 分治

left right

① 确定分界点: $mid = (l+r)/2$

② 递归排序 left right

③ 归并 ——— 合 = 为一 ☆



(3)、最近点对, 好题, 好题, 值得细品!!!

solve(l, r) 做两件事:

1. 求出 $p[1 \sim r]$ 这个点集内部的最近距离
2. 把 $p[1 \sim r]$ 按 y 坐标排好序

$p[]$ 这个数组在算法过程中，它的“排序意义”发生了变化：

一开始： $p[1 \sim n]$ 按 x 排序

递归过程中： $\text{solve}(l, r)$ 返回后， $p[l \sim r]$ 被改成按 y 排序

这看起来很怪，但它是为了把合并阶段做到 $O(n)$ 。

PPT 里的伪代码也强调：递归函数返回最近距离，同时把当前区间按 y 排序；
临界区 Y_d 不需要重新排序，只需要扫描一遍得到。

二、DP

化零为整：状态统一表示

化整为零：根据最后一步的操作划分集合

分治vsDP:

分治是把问题拆成互不重叠的子问题，分别求解再合并；DP 是把问题拆成会重复出现的子问题，把子问题答案存起来，通过状态转移得到最终解。

递归、自底向上都只是实现方式，不是本质区别。

核心！闫式dp分析法嘿嘿嘿：

集合定义的核心！：每一个选法都能对应到状态表示的某一个维度！有泛化性，每一个最优子结构均能包含进去！

```
//状态表示：f[i][j].....
```

```
//集合定义
```

```
//属性：Max, Min, 数量
```

```
//状态计算
```

```
//集合划分（化整为零，对当前集合进行划分，能否重合依据题意）
```

```
//
```

```
//状态转移方程（从前往后递推关系式）
```

二一、背包问题(特殊的线性dp：沿着物品编号和容量推进)

在共n种物品中选，使得容量为m的背包所装价值最大

均为第i-1层推出第i层

1、01背包小结（每种只能选一个，物品不重复）

答案：f[n][m]

```
n = 4, m = 5

物品1: v=1, w=2
物品2: v=2, w=4
物品3: v=3, w=4
物品4: v=4, w=5
```

分类（对f[i][j]的定义不同导致不同的初始化）

1.1容量上限型dp

f[i][j]:前i种物品中选，总体积<=j

```
f[0][0]=0
f[0][1~m]=0
```

```
f[0]: 0 0 0 0 0 0
f[1]: 0 2 2 2 2 2
f[2]: 0 2 4 6 6 6
f[3]: 0 2 4 6 6 8
f[4]: 0 2 4 6 6 8
```

1.2精确体积型dp

f[i][j]:前i种物品中选，总体积=j

```
f[0][0]=0
f[0][1~m]=-INF
```

```
f[0]: 0 -INF -INF -INF -INF -INF
f[1]: 0 2 -INF -INF -INF -INF
f[2]: 0 2 4 6 -INF -INF
f[3]: 0 2 4 6 6 8
f[4]: 0 2 4 6 6 8
```

代码1：二维写法f[i][j]

```
for(int i=1;i<=n;i++)
{
    for(int j=0;j<=m;j++)
    {
        f[i][j]=f[i-1][j];//
        if(j>=v[i])f[i][j]=max(f[i-1][j],f[i-1][j-v[i]]+w[i]);
    }
}
cout<<f[n][m];
```

代码2：一维写法f[j]，逆序枚举j，此时的f[j]还没有被更新过，还是上一层（i-1）的f[j]

```
//优化：每一层i仅于第i-1层有关
//使用一维滚动数组，每次只保留i-1层，用于更新未i层
for(int i=1;i<=n;i++)
    for(int j=m;j>=v[i];j--)
        f[j]=max(f[j],f[j-v[i]]+w[i]);
cout<<f[m];
```

正序枚举是错的

```
for(int i=1;i<=n;i++)
    for(int j=v[i];j<=m;j++)
        f[j]=max(f[j],f[j-v[i]]+w[i]);
cout<<f[m];
```

2、完全背包小结，物品可重复，无限个（只要不超过背包体积）

一维写法f[j],正序枚举j

```
for(int i=1;i<=n;i++)
    for(int j=v[i];j<=m;j++)
        f[j]=max(f[j],f[j-v[i]]+w[i]);
cout<<f[m];
```

3、多重背包，物品可重复，每种最多s个

暴力 $O(n^3)$

4、多重背包二进制优化->01背包

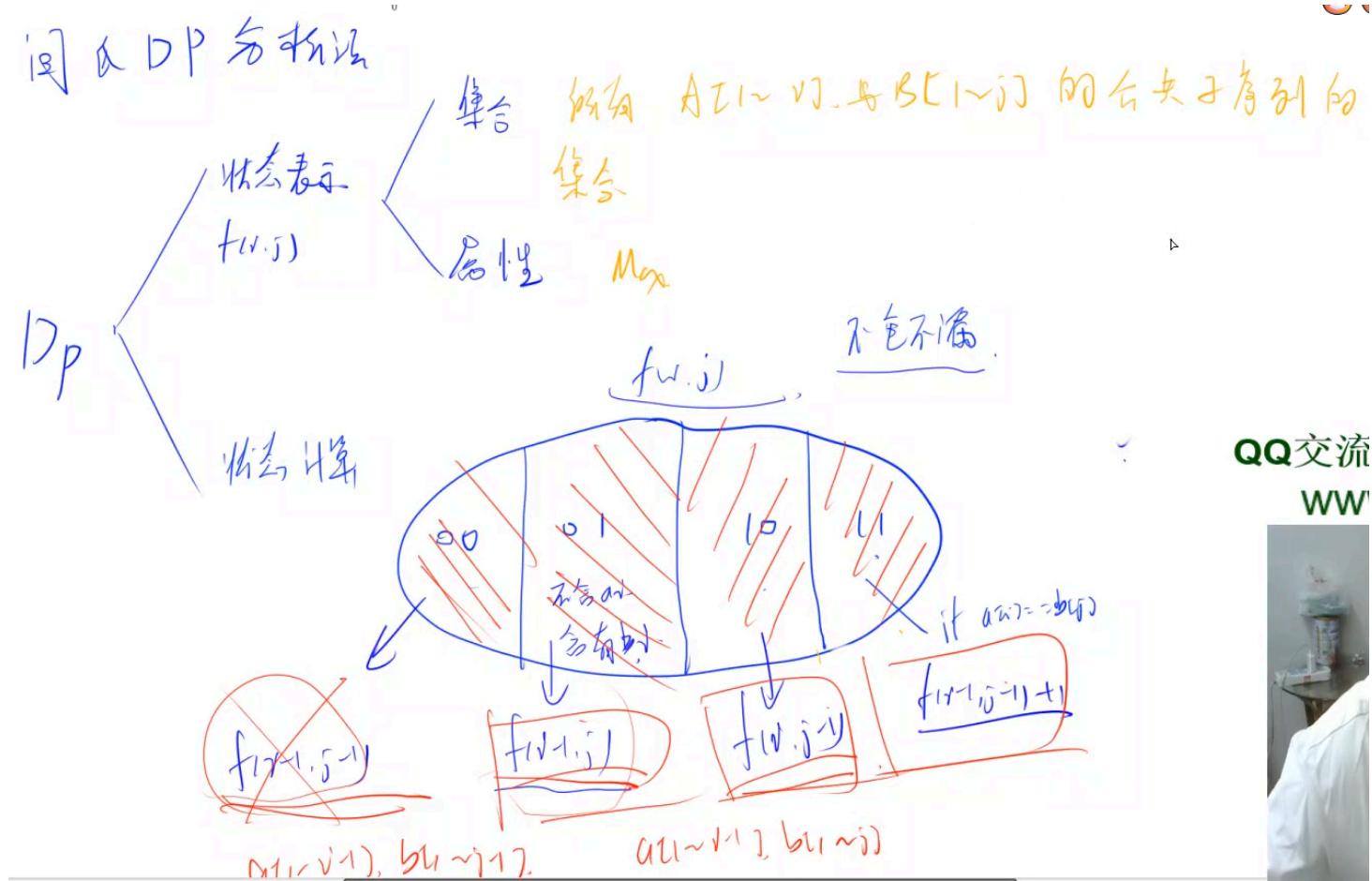
每个s采用二进制拆分， $9=1+2+4+2$ 个小包裹，之间任意排列组合便可得到0~9，故原本朴素（每种物品视为s个独立物品）优化为 $\log n$ 个

$O(n^2 \log n)$

5、分组背包：(01背包可以视作分组背包的特例，每组中所有物品均相同且只能选一样)

二二、线性dp（沿着一个序列或长度推进）

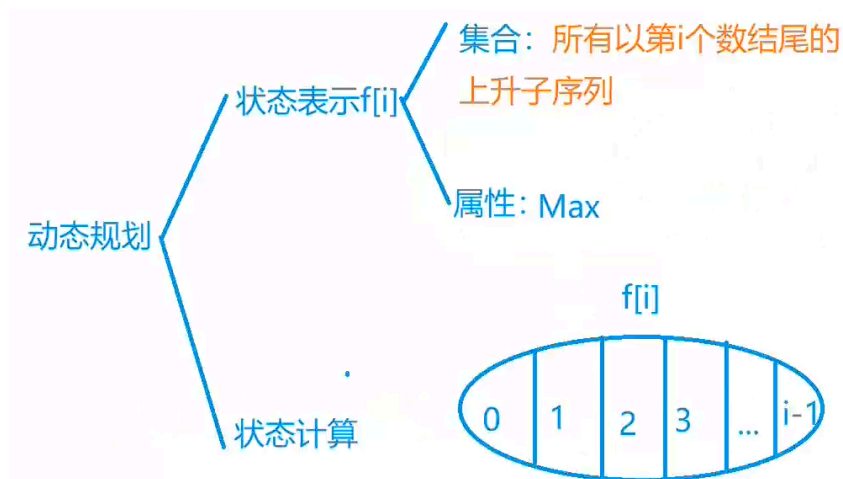
1、最大公共子序列：



注意，此题的集合划分之间有重叠部分，因为最后是求最大值，所以可以重复，如果是求和就不能这样划分了

所以只是类似全概率公式，但有些题也没有这么严格的要求

2、(非降)子序列 I: (dp, O(n^2))



7
3 1 2 1 8 5 6

www.acwing

3、最大乘积，线性枚举最后一个乘号出现的位置

```
//随时警惕下标哈哈哈哈哈
for(int i=2;i<=n;i++)//枚举整个序列终点i的位置；枚举从2~i的范围，等价于枚举长度1~i
    for(int j=1;j<=k;j++)//枚举1~i这个序列中*的个数，为j
        for(int t=j;t<=i-1;t++)//1~t里有j-1个*(f[t][j-1])，在t后面有一个*，*value[t+1][i];
            //枚举1~i这个序列的最后一个*的位置（出现在哪个数后面）
            f[i][j]=max(f[i][j],f[t][j-1]*value[t+1][i]);
            //所以必须每次都清零，应为max(先前上一组数据的f[i][j],)
cout<<f[n][k]<<endl;
```

4、钢条切割（本质是完全背包问题）

容量 = 钢条长度 n
物品 = 可卖钢条长度 $l[i]$
价值 = 价格 $p[i]$
每种物品可用无限次

5、最小票价，日票、周票、月票（暴力线性枚举可利用有序性优化为二分查找或双指针）

法一， $O(30*n) \sim O(n)$, $n < 365$;

暴力从 i 往前扫（最多30个），找第一个不被票覆盖的位置。

法二： $O(n)$

基于 $days[i]$ 序列， i 、 j 指针移动的单调性，可以用双指针优化

法三： $O(n \log n)$, 此题差不多

基于 $days[i]$ 序列值与下标的有序性（通常为值的单调性，但二分的本质是性质），可以使用二分查找最大的不被票覆盖的位置。

总结：二分法vs双指针

在有序性序列中搜索某值，除了用暴力线性枚举，可能可以二分或双指针，可能都能用，可能只能用一种，因为基于的性质不同

双指针一般是用于 n^2 利用指针单调性优化到 n ,此题不明显，但需要有这个充分利用性质的优化复杂度的思想

6、带权活动选择（最大不相交区间数量的plus版，贪心），和最小票价的递推关系特别像！

先按区间右端点排序，这样做的目的不是贪心地选最早结束的活动（最大不相交区间数量），而是为了让我们能定义：

$dp[i]$:只考虑前 i 个活动时，能获得的最大价值。

这问题有了清晰的“前缀结构”。

真正的贪心是每一步直接做一个局部最优选择，不回头。

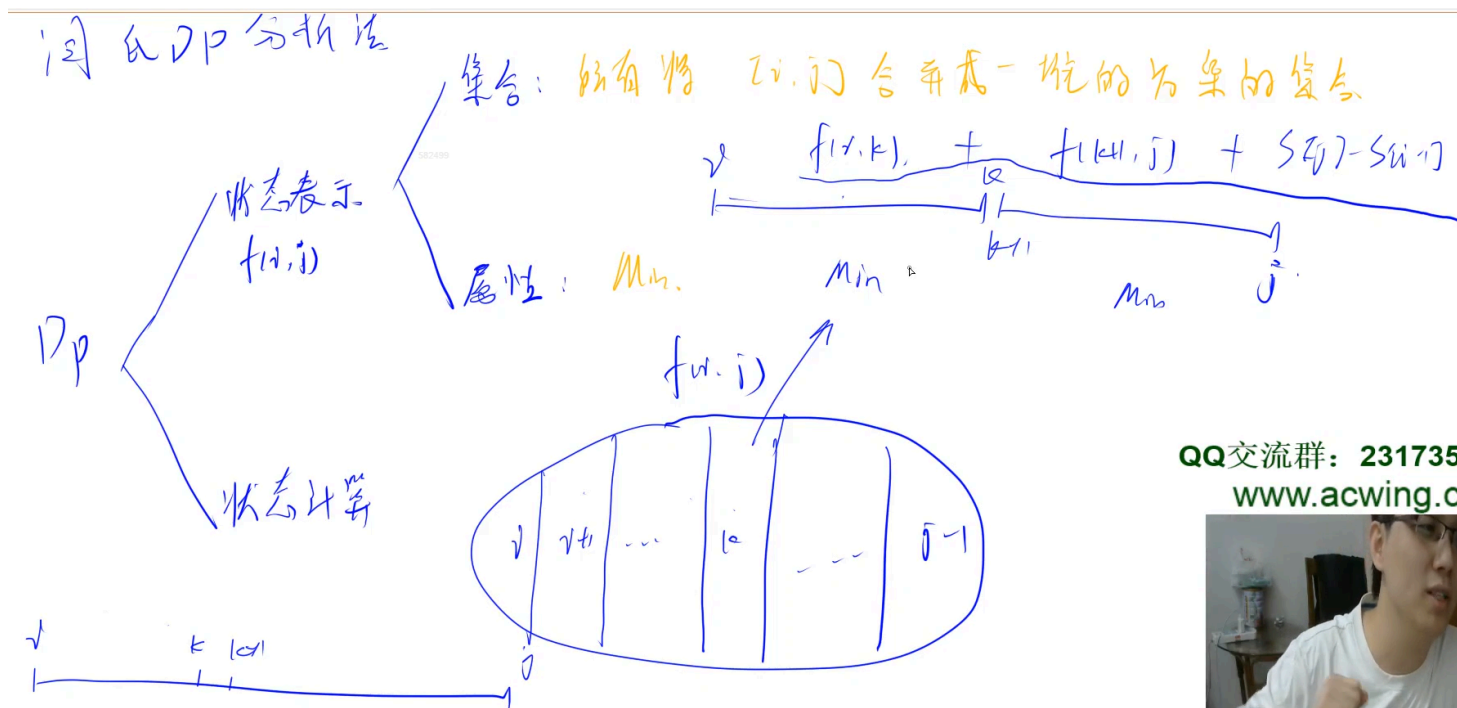
```
//1、不包含活动i
//2、包含活动i，利用值与下标的单调性，
//二分法找到最后一个不重叠的区间(找前驱)
    //k为最后一个终点ranges[k].f<=ranges[i].s的，
    //最大数k
int k=find(0,i-1,i);
dp[i]=max(dp[i-1],dp[k]+ranges[i].v);
```

二三、区间dp（沿着区间长度推进）

1、石子合并（每次合并“相邻”堆，最小体力）

由于“相邻”所以破坏了贪心性质，不能够每次合并和最小的相邻堆，有反例，

vs合并果子（贪心，不要求相邻，所以没有破坏贪心性质，每次将最小的两堆合并即可），采用priority_queue,声明为最小堆即可



2、插入乘号

不能按照左右边界 i, j 来枚举，因为最后一步是将两个短区间合并为一个长区间

要先从小到大枚举所有区间长度，再枚举起点

$f[1][5]$ 会用到 $f[4][5]$ ，不然的话只枚举了从1开始的，从4开始的就还没有

$f[i][j][k]$: $i \sim j$, 共 k 个乘号

$f[1][n][k]$

3、矩阵连乘

看最后一步，已定是由左右两个段区间合并而来的，枚举最后一次乘法的间断点

4、最优二叉搜索树

```
//每棵二叉树又左子树、根节点、右子树构成，
//最优子结构：左右子树均最佳

//给定N个整数关键字，编号1~N，则还有N+1个空隙区间，编号0~n；
//最终问题是构建一颗最优的二叉搜索树，使其具有最优的结构，使得总代价最小
//与上司的树形dp不同（树结构已定，根据父子关系推进），
//此题采用区间dp：树的结构未定，需要自己构造最优的，且重叠的子问题为一段一段的区间，
//区间长度从小到大枚举

//求f[0][n],key[1~n],p[1~n],q[0~n],w[0~n][0~n]
//(w[i][j],第i、j个空隙和其间所有key构成的权值和，用于状态转移时的均下沉一层操作)

//初始化:
//w[i][i]=q[i],f[i][i]=0(只有一个空间隙，无key，连树都还没有构成，何谈代价@@@)
//w[i][j]=w[i][j-1]+p[j]+q[j];(j>=i+1)
//状态表示:
//集合：f[i][j]空隙 q[i]~q[j] 之间夹着的那些key (key[i+1],key[j])，所有树形结构
//属性：min，最小代价
//集合计算
//集合划分:
//从左至右枚举根节点，
//状态转移方程：枚举根节点k号key，k:i+1~j，
//左子树：i+1~k-1号key
//root:k
//右子树：k+1~j;
//f[i][j]=(f[i][k-1]+w[i][k-1])+p[k]+(f[k][j]+w[k][j])
//f[i][j]=min(f[i][j],f[i][k-1]+f[k][j]+w[i][j]);
```

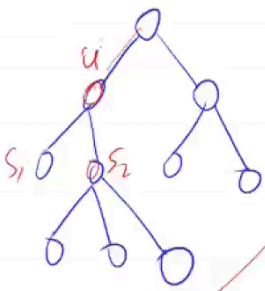
二四、树形dp（沿着树的父子关系推进）

线性 DP、区间 DP、背包 DP 常常同时有明显的最优子结构和子问题重叠；
树形 DP 更强调最优子结构，子问题重叠性

1、没有上司的舞会

$$f(u, 1) = \sum f(s, 0)$$

2N
O(N)



$$f(u, 0) = \sum \max(f(s, 0), f(s, 1))$$
$$f(u, 1) = \sum f(s, 0)$$

$$f(u, 0) + \max(f(s, 0), f(s, 1))$$

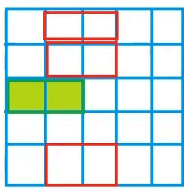
www.acw



二五、状态压缩dp

1、蒙德里安的梦想：

i-2 i-1 i



i=2, j=11001

动态规划

状态表示: $f[i, j]$ 表示已经将前 $i-1$ 列摆好, 且从第 $i-1$ 列, 伸出到第 i 列的所有方案。

的状态是 j

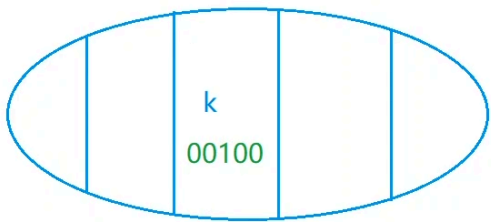
核心:

先放横着的, 再放竖着的。

总方案数, 等于只放横着的小方块的合法方案数。

如何判断, 当前方案是否合法? 所有剩余位置, 能否填满竖着的小方块。可以按列来看, 每一列内部所有连续的空着的小方块, 需要是偶数个。

状态计算:

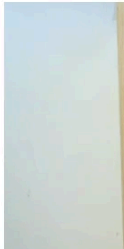


$f[m, 0]$

(1) $(j \& k) == 0$

(2) 所有连续空着的位置的长度必须是偶数

www.



二六、能力型dp（上面多数为策略型dp）

鸡蛋掉落（蛋疼）

经过 2h 的战斗，总结如下：

此题本质：能力型dp，二叉决策树

传统做法：

$f[k][n]=m, O(k \cdot N^2)$, 二分可至 $k \cdot N \log N$,
 $dp[k][n] = 1 + \min_x \max(dp[k-1][x-1], dp[k][n-x])$

改用能力型状态定义：->反过来做，资本为k、m，能力为覆盖f[k][m]层楼（连续区间长度）

$f[k][m]=n$

将求f[k][m]这个大问题进行拆分，通过已经解出的子问题合并得到大问题的解。

- 怎么划分大问题呢？

选择第一次扔的那个楼层作为分界点， $f[k][m]=\text{左区间}+1+\text{右区间}$ ，

第一次扔的楼层是分界点。

碎了，就进入左区间；没碎，就进入右区间。

注意！！！！

实际执行时只会走其中一个分支，

但策略设计时必须保证两个分支都能处理。

- 怎么计算左右区间长度呢？

直接用状态定义来表示，

同时注意到：存在左区间是有前提的，就是在分界点这一层仍必须是碎了，才能在左区间去找F，

比如说分界点为0层，这一层不碎，就不存在左区间，只有右区间，

那么左区间最长有多长呢？ $f[k-1][m-1]$ ，

好，再算右区间，存在右区间的条件：分解点仍不会碎，才会继续往上查找F， $f[k][m-1]$ 。

所以，计算左右区间是相互独立并行的，不存在说右区间（没碎）在左区间（碎了）确定的floor的基础上+ $f[k][m-1]$ 的说法，

仅仅是用状态表示出子问题的数值是多少，两段区间相加罢了

- 刚好碰巧分界点的位置可以由左区间确定，右区间从这个分界点的上方开始继续拼接。

二七、计数类dp（总方案数目）

1、整数划分：

法一：完全背包的思路

$4=3+1$
 $=2+2$
 $=2+1+1$
 $=1+1+1+1$

背包总容量为4，共4种物品，体积分别为1，2，3，4
 同种物品可重复，要求恰好装满

状态定义：f[i][j]：

集合：前i种物品中选（第1~i种物品体积对应1~i），总体积为j的所有方案

属性：方案数量（传统完全背包为max价值）

状态计算：

状态划分：

$f[i][j] = f[i-1][j-i] + f[i-1][j-2*i] + \dots + f[i-1][j-s*i]$

暴力为 $O((n^2)\log n)$

进行类似完全背包的优化：

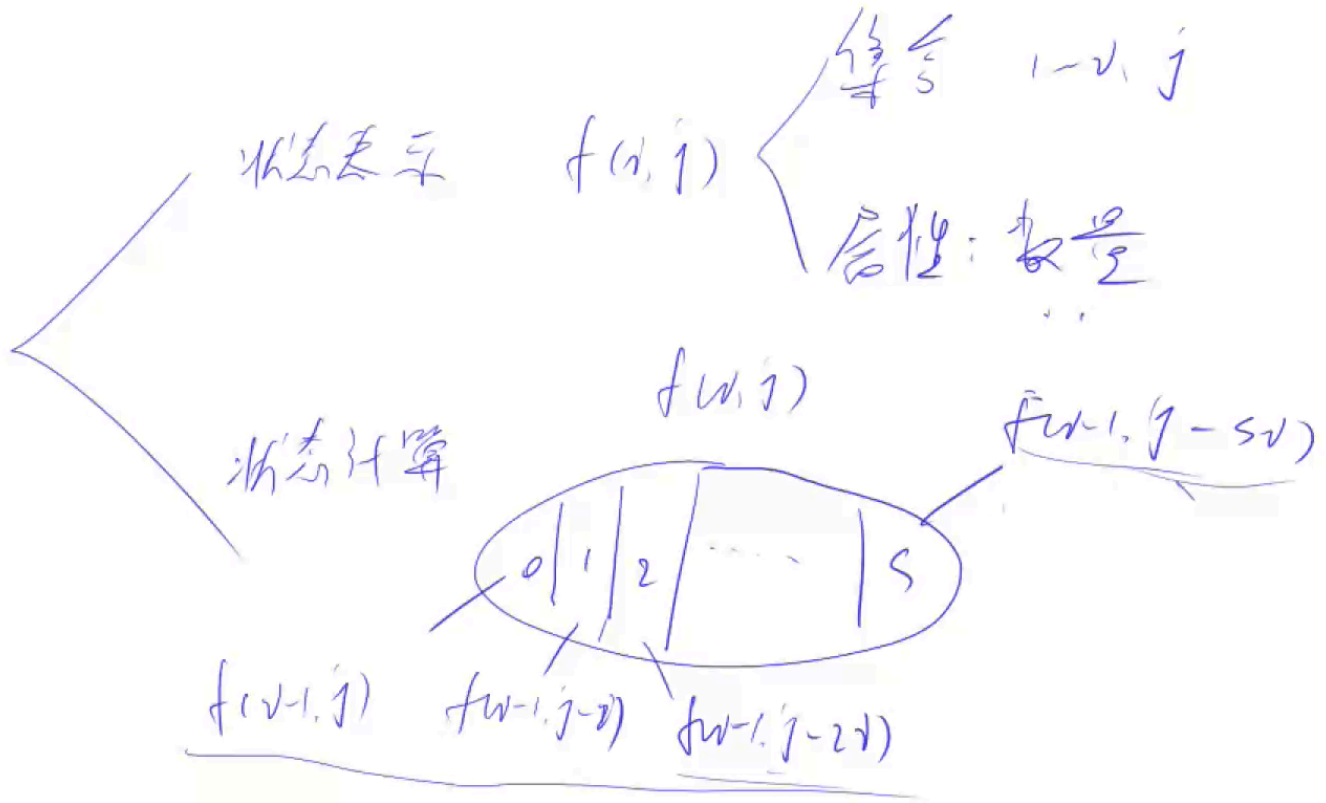
$f[i][j-i] = f[i-1][j-2*i] + \dots + f[i-1][j-s*i]$

->

$f[i][j] = f[i-1][j-i] + f[i][j-i]$

$O(n^2)$ ，把枚举的中间变量i的个数k抵消了

$$\frac{1}{1}, \frac{1}{2}, \dots, \frac{1}{1000}, \log n$$



f[0]: 1 0 0 0 0 0

f[1]: 1 1 1 1 1 1

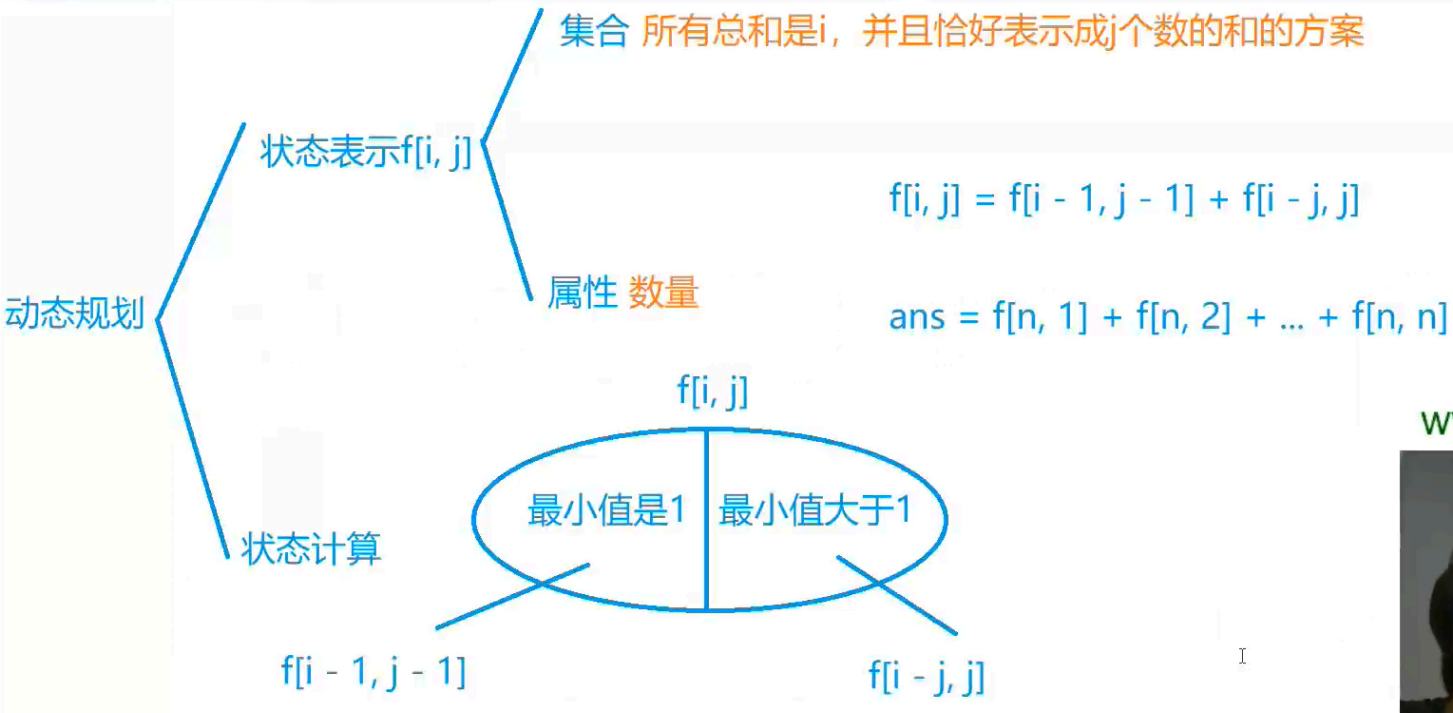
f[2]: 1 1 2 2 3 3

f[3]: 1 1 2 3 4 5

f[4]: 1 1 2 3 5 6

f[5]: 1 1 2 3 5 7

法二：另一个奇妙视角，神奇的状态定义



2、装饰

三、贪心

1、区间选点

数轴上选择尽量少的点，使得每个区间内至少包含一个选出的点。

<=>最大不相交区间数量（活动选择问题）

在数轴上选择尽可能多的区间，使得选中的区间之间互不相交（包括端点）。

法一、按左端点排序 + 动态收缩右边界

- `st`、`ed` 记录当前的交集区间，随时更新；
- 包含加入的区间时，更新为新的最小完整交集区间
- 当不包含加入的区间时，结果加1，区间变为新区间


```

int cnt(vector<PII>&segs)
{
    int res=0;
    int st=-2e9,ed=-2e9;
    for(auto seg:segs)
    {
        if(ed<seg.first)
        {
            res++;
            st=seg.first;//此行不更新也不影响结果
            ed=seg.second;
        }
        st=seg.first;//此行不更新也不影响结果
        if(seg.second<ed)ed=seg.second;
    }
    return res;
}

```

法二、按右端点排序 + 遇到不包含的新区间时计数加一（每次选择最早结束的活动）

① $Ans \leq cnt$

② $Ans \geq cnt$ ✓

$\Rightarrow Ans = cnt$

1. 将每个区间按右端点从小到大排序

2. 从前往后依次枚举每个区间

 如果当前区间中已经包含点，则直接pass

 否则，选择当前区间的右端点

1.1 区间选点时的贪心证明

(选择尽量少的点，使得每个区间内至少包含一个选出的点)

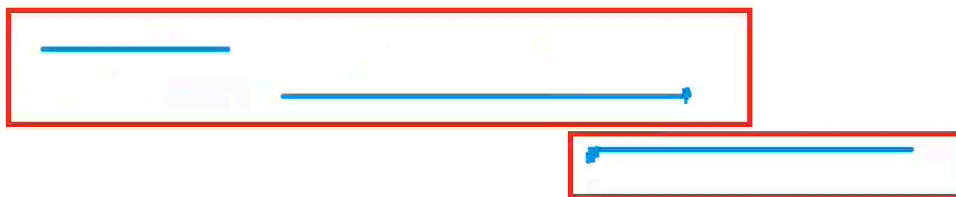
- $Ans \leq cnt$, 显然;
- $Ans \geq cnt$, cnt 个点覆盖了 cnt 个独立的区间, 想要覆盖所有的区间, 则 $Ans \geq cnt$;

1.2 最大不相交区间数量的贪心证明:

- 已找出 cnt 个不相交的子区间, 且这 cnt 个区间的 cnt 个右端点能覆盖所有区间, $Ans \geq cnt$, 显然;
- 欲证 $Ans \leq cnt$, 反证法:
 - 假设 $Ans > cnt$, 最多有 Ans 个不相交子区间, 至少要 Ans 个点才能覆盖所有区间。
 - 这与仅需 cnt 个点就可覆盖所有区间矛盾!

2、区间分组

每组内部的区间两两之间 (包括端点) 没有交集, 并使得组数尽可能小



1. $Ans \leq cnt$ ✓

2. $Ans \geq cnt$

$ans == cnt$

1. 将所有区间按左端点从小到大排序

2. 从前往后处理每个区间

判断能否将其放到某个现有的组中 $L[i] > Max_r$

1. 如果不存在这样的组, 则开新组, 然后再将其放进去;
2. 如果存在这样的组, 将其放进去, 并更新当前组的 Max_r

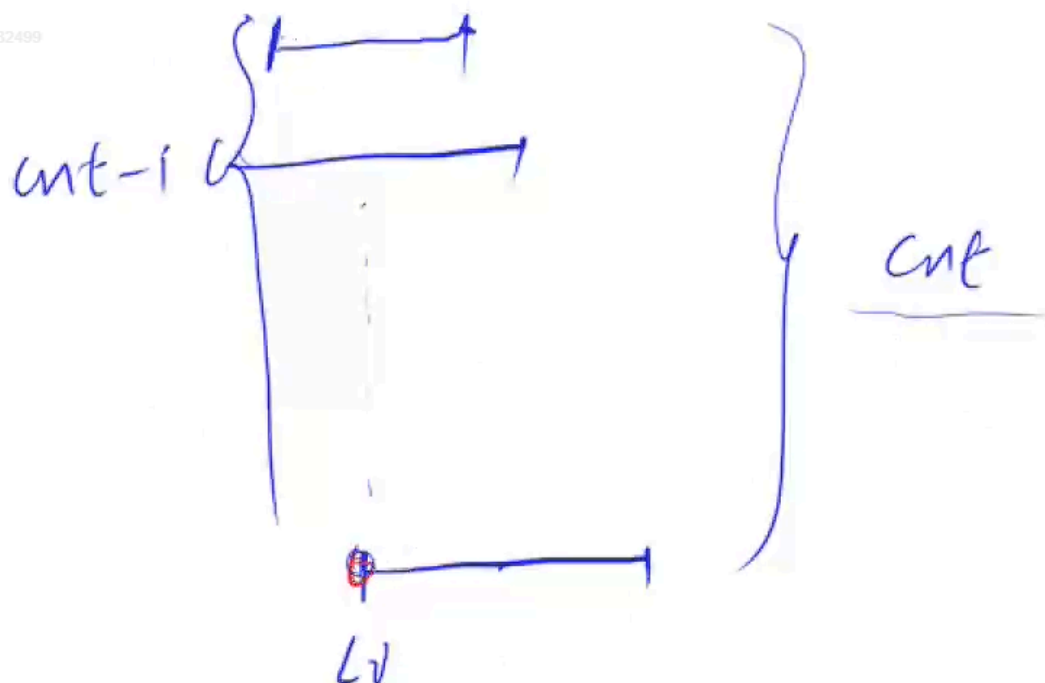
www.acv



cnt max-r

$max \geq L_v$

582499



3、区间覆盖

选择尽量少的区间，将指定区间完全覆盖。

start  end

1. Ans $\leq$ cnt ✓

2. Ans $\geq$ cnt



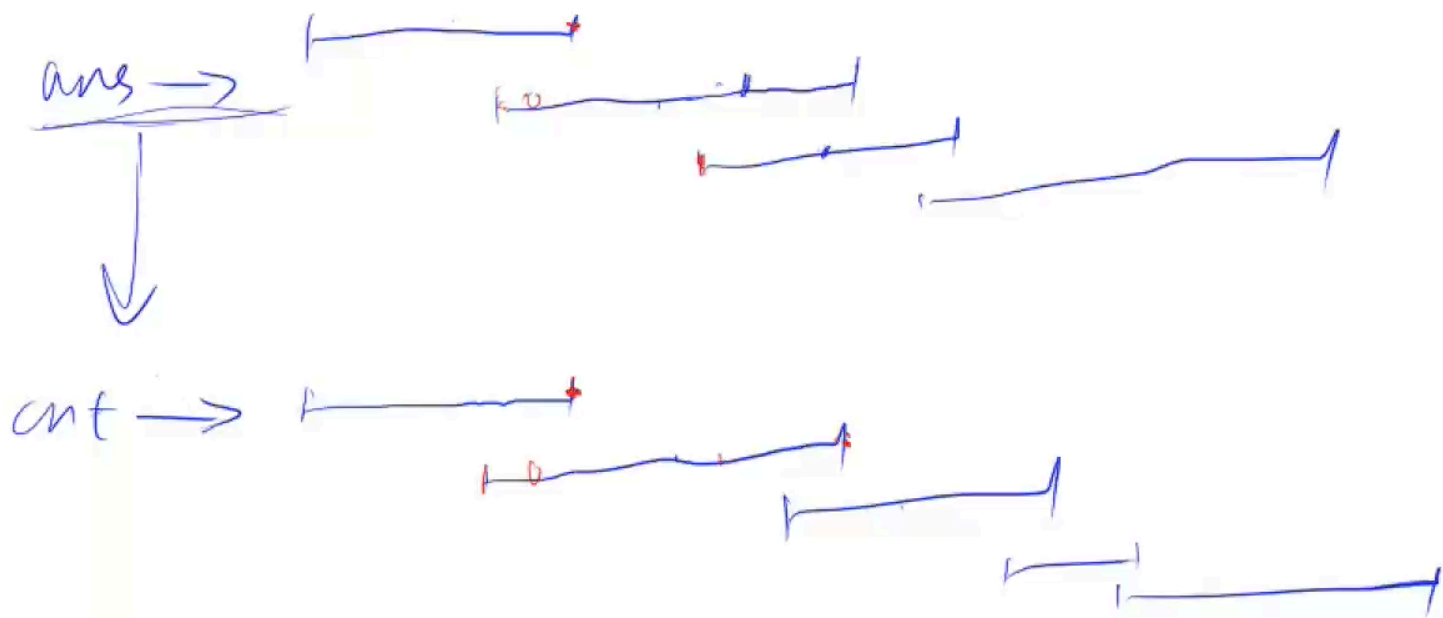



Ans == cnt

1. 将所有区间按左端点从小到大排序

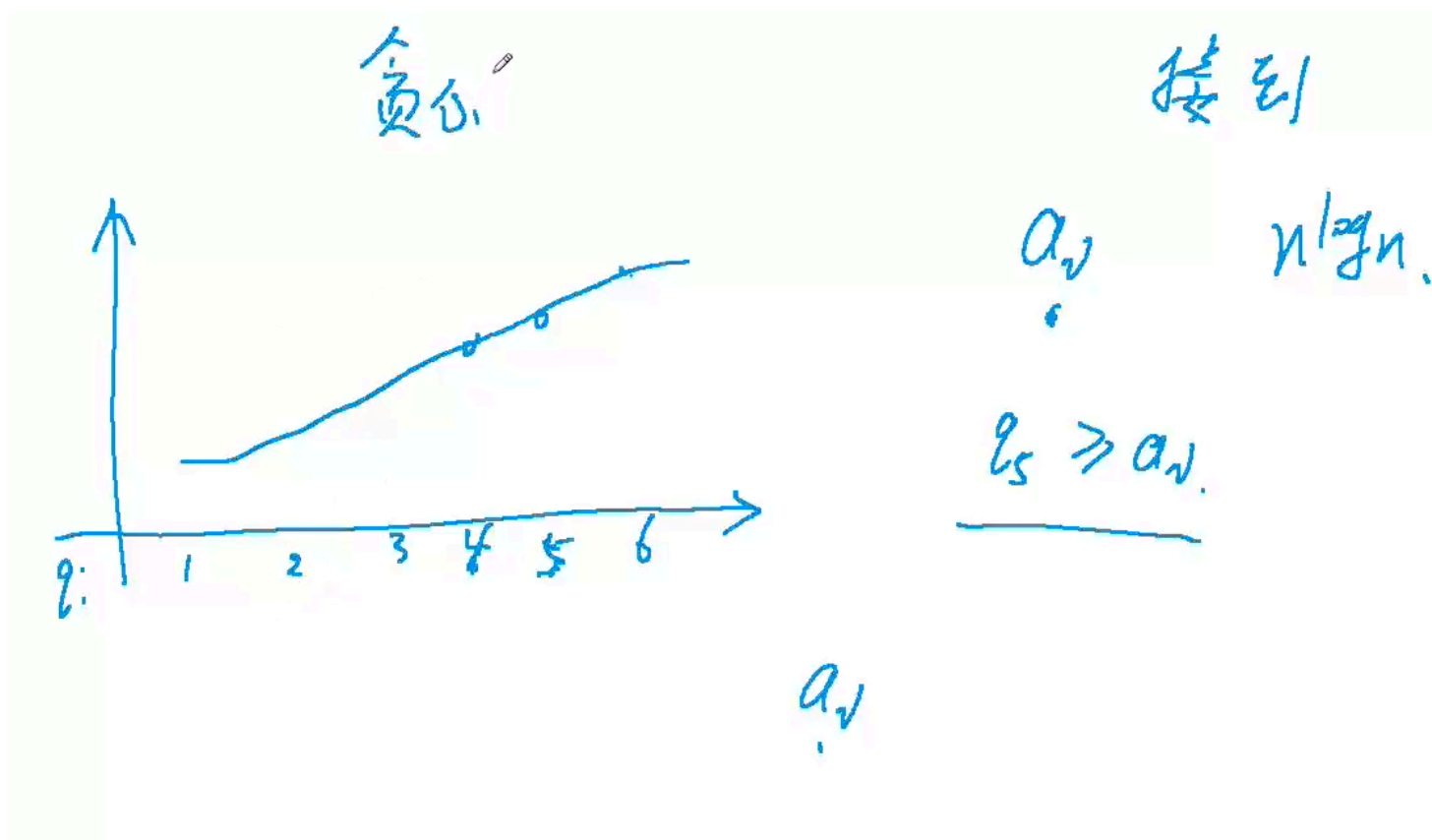
2. 从前往后依次枚举每个区间，在所有能覆盖start的区间中，选择右端点最大的区间
然后将start更新成右端点的最大值

www.acv



对于任何一个最优解，都能等价地替换为算法选出的方案，则 $ans==cnt$

4、最长上升(非降)子序列 II (贪心, $O(n \log n)$)



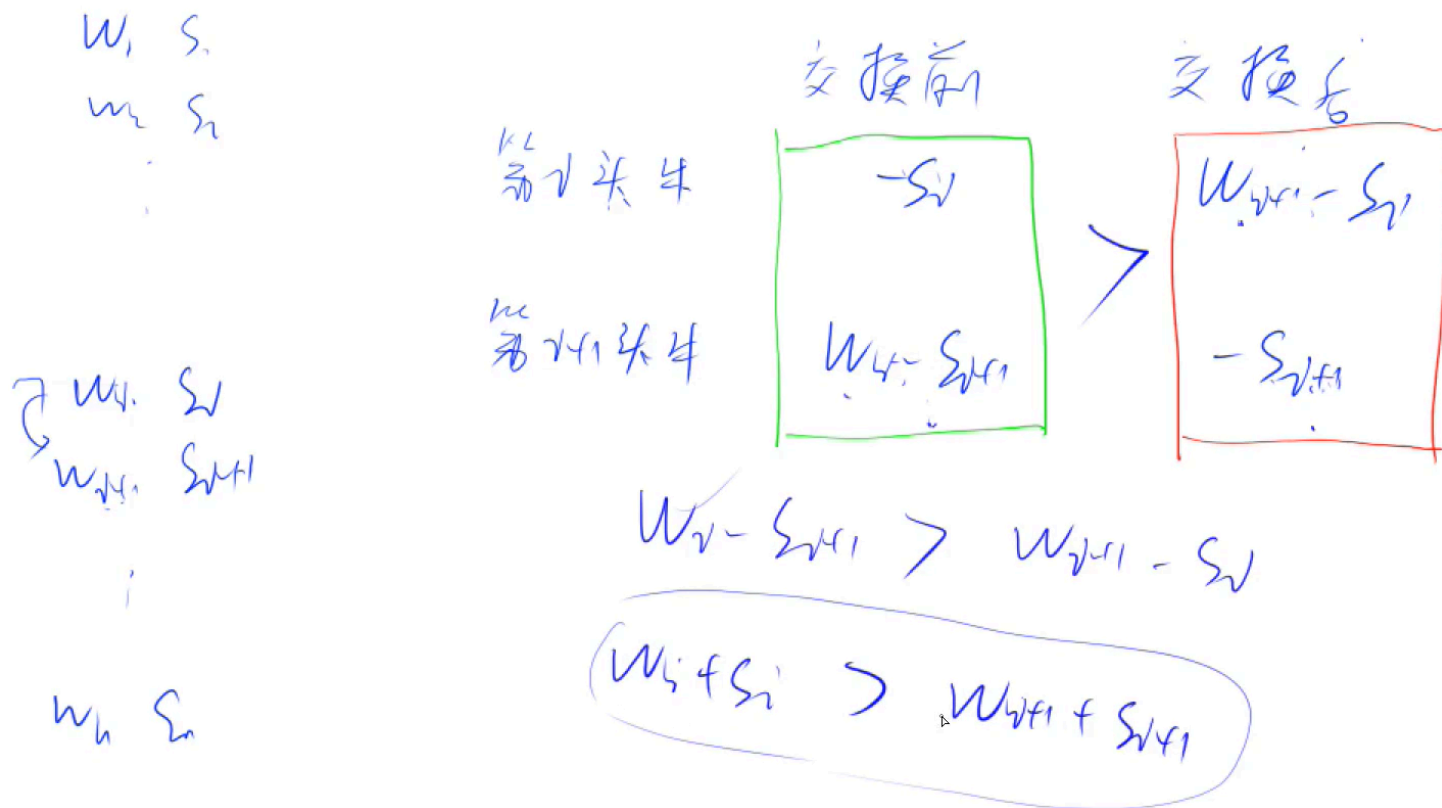
5、耍杂技的牛：

先直观感知， w 越大， s 越大，应该放在越下面；

那具体怎么刻画这个“大”呢？

先随便假设一个顺序，考虑其中相邻两牛交换前后对这两个牛的影响（由于相邻对上下均无影响），发现 $w_i + s_i$ 不能存在逆序即可！

即 $w_i + s_i$ 全部正序排列！



6、黑白连线

贪心+双指针优化 (i 、 j 下标的单调性,随着 i 的增大, j 永不回退)

每次取最左边还没匹配的点 i ，把它和右边第一个还没匹配且颜色相反的点 j 匹配。

要证明它最优，核心是证明：

这个 i 和这个 j 匹配，一定不会使答案变差。

贪心证明：

3. 交换证明

考虑任意一个最优方案 OPT 。

如果在 OPT 中, i 已经和 j 匹配, 那就没问题。

如果没有, 那么在 OPT 中:

```
</> C++  
  
i 和 q 匹配  
j 和 p 匹配
```

其中:

```
</> C++  
  
color[i] = color[p] = c  
color[j] = color[q] = 1 - c
```

因为 j 是 i 右边第一个异色点, 所以 q 一定在 j 的右边:

```
</> C++  
  
i < j < q
```

而 p 是和 j 匹配的同色点, 它可能在 j 左边, 也可能在 j 右边。

原方案里有两条匹配:

```
</> C++  
  
(i, q)  
(p, j)
```

现在我们把它们改成:

```
</> C++  
  
(i, j)  
(p, q)
```

颜色仍然合法, 因为:

```
</> C++  
  
i 和 j 异色  
p 和 q 异色
```

所以新方案仍然是一个合法匹配方案。

4. 证明交换后距离不会变大

分情况讨论。

情况一: $p < j < q$

原来的距离是:

```
</> C++  
  
(q - i) + (j - p)
```

交换后的距离是:

```
</> C++  
  
(j - i) + (q - p)
```



情况一: $p < j < q$

情况二： $j < p < q$

情况三： $j < q < p$

5. 得出结论

无论 p 在哪里，我们都能把最优方案改造成一个新的最优方案，并且让：

</> C++

i 和 j 匹配

也就是说：

贪心选择的这一步一定可以出现在某个最优解中。

这就证明了贪心第一步是安全的。

6. 递归到剩下的问题

匹配完 i 和 j 后，把这两个点删除。

剩下的点仍然是同样的问题：

</> C++

若干个 0 和若干个 1，继续两两异色匹配，使总距离最小。

所以可以对剩下的点继续使用同样的贪心策略。

由数学归纳法：

</> C++

每一步贪心选择都可以出现在某个最优解中

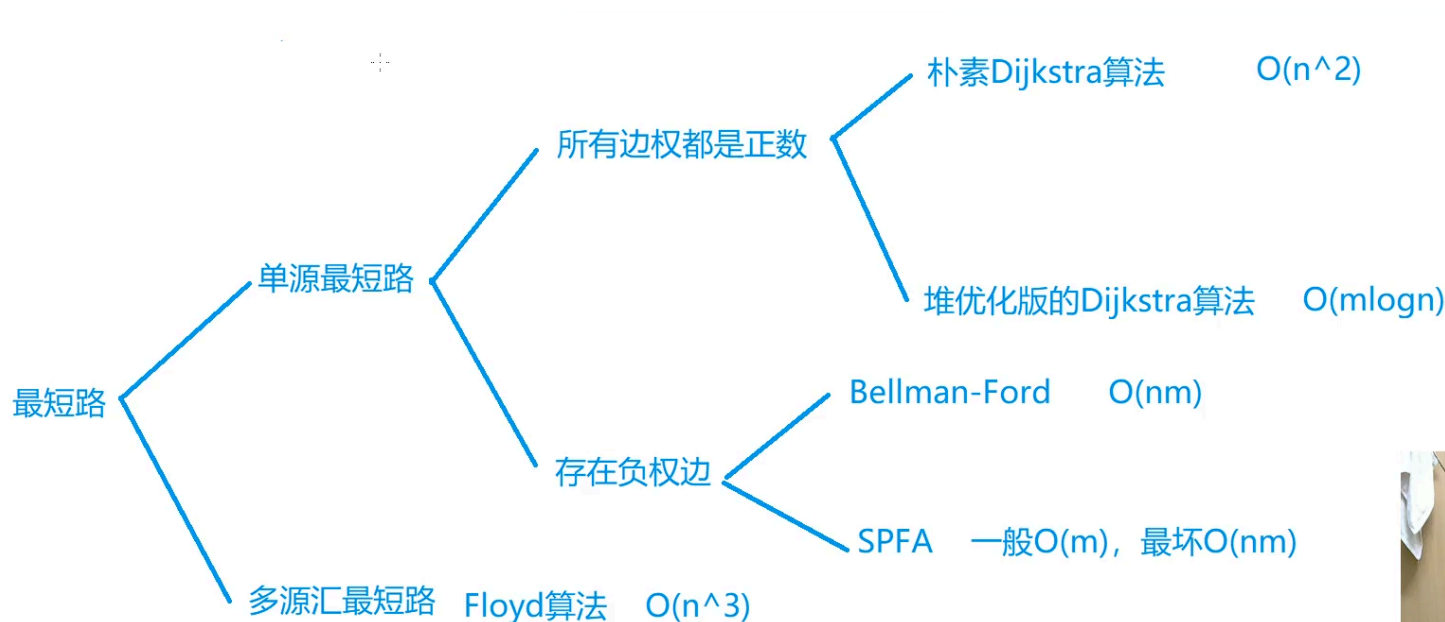
↓

因此整个贪心算法得到的总距离就是最优解。

五、图论：

1最短路：(一圈一圈往外拓展)

1.1dijkstra (贪心) (边权均 >0):



1.1.1朴素dijkstra: $O(n^2)$ 适用稠密图, 邻接矩阵

朴素 Dijkstra

S : 当前已确定最短距离的点

① $dist[i] = 0$, $dist[i] = +\infty$

② for $v: 1 \sim n$

$t \leftarrow$ 不在 S 中的, 距离最近的点

$S \leftarrow t$

用 t 更新其它点的距离

关闭

www.acwing.com



1.1.2堆优化dijkstra: $O(m \log n)$, n 为点数, m 为边数, 适用于稀疏图, 对于稠密图会退化为 $(n^2) \log n$, 邻接表 (重边不影响)

嘿嘿嘿😁
密图也一样

好耶

哈哈

$v: 1 \sim n$

$t \leftarrow$ 不在 S 中的, 距离最近的点

n

$S \leftarrow t$

n 次

用 t 更新其它点的距离

m 次

堆 < 手写堆 n 次

优先队列 m 次

$m \log n$

$m \log n$

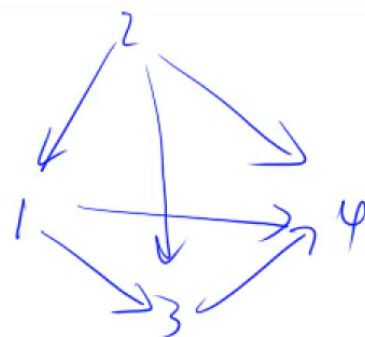
优先队列不支持修改任意元素, 会出现冗余, $st[]$ 记录之前是否被扔掉, 扔掉就直接continue

有向图、邻接矩阵

$a \rightarrow b$

邻接表

$h[1] \rightarrow 0 \rightarrow 0 \rightarrow \phi$
 $h[2] \rightarrow 0 \rightarrow 0 \rightarrow \phi$
 $h[3] \rightarrow 0 \rightarrow \phi$
 $h[4] \rightarrow \phi$



$2 \rightarrow 3$

www.acw



1.1.3 隐式图的堆优化dijkstra：穷游！，（费用限制的最短路问题）

隐式图中每个节点状态为（编号，cost）

使用剪枝加速算法（第一次弹出t，直接break），减少不必要的计算

1.2 Bellman-Ford（

dp）（边权有 <0 ，结构体任意顺序随便存边）

Bellman - Ford 算法



for n次

for 所有边 a, b, w

松弛操作

$$dist[b] = \min(dist[b], dist[a] + w);$$

dist 1 2 3
0 ~~∞~~ ∞
1

$$dist[b] \leq dist[a] + w$$

三角不等式

www.acw



再从数学上说一遍

定义:

$$f[t][v] = \text{从1到} v, \text{最多经过} t \text{条边的最短距离}$$

那么:

初始状态

$$f[0][1] = 0$$

$$f[0][v] = INF \quad (v \neq 1)$$

转移

如果有边 $a \rightarrow b$, 边权为 c , 那么:

$$f[t][b] = \min(f[t][b], f[t-1][a] + c)$$

意思是:

- 要用不超过 t 条边到 b
- 最后一条边可能来自 a
- 那前面必须先用不超过 $t-1$ 条边到 a

你的代码其实就是这个 DP 的滚动数组版:

- `last[a]` 对应 $f[t-1][a]$
- `dist[b]` 对应 $f[t][b]$

最短路径Bellman-ford算法

BELLMAN-FORD(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 **for** $i \leftarrow 1$ **to** $|V[G]| - 1$

3 **do for** each edge $(u, v) \in E[G]$

4 **do** RELAX(u, v, w)

5 **for** each edge $(u, v) \in E[G]$

6 **do if** $d[v] > d[u] + w(u, v)$

7 **then return** FALSE

8 **return** TRUE

N-1次迭代后
(可以最多经过
n-1条边) 还能
继续松弛, 有负
环

1.2.1: 有边数限制的最短路: 外层while k次 (完完全全的一维滚动数组dp)

1.2.2: 常规的最短路: 就直接外层while循环n-1次就ok~

1.3 spfa (边权 <0 ,)

for 所有边 a, b, w

$dist[b] = \min(dist[b], dist[a] + w);$

queue ← 1

while queue 不为空

① $t \leftarrow \text{queue.front};$

$\text{queue.pop}();$

② 更新 t 的所有出边 $t \rightarrow b$

queue ← b

结构和BFS很像，但BFS是无权图寻最短路，一圈一圈往传播的层数（等权，距离渐远传播），spfa也是类似的波纹传播，波纹扩散的是更新最小距离，

1.3.1 spfa求最短路

1.3.2 spfa判断负环：

(cnt[i]表示:当前把点i更新成功时，这条最短路经过了多少条边)

如果某条变优路径长度已经达到 n 条边，说明它绕了一个还能让答案变小的环，也就是负环。)

与spfa寻最短路相比，一定要把dist全部初始化为

0，因为没有限制一定要1是源点，这样会限制更小的搜索范围，这也是为什么初始队列装入所有点，保证所有其他点都能访问到不会漏过负环

1.4 Floyd (是dp，不是区间dp)：(多源汇最短路，含负边，不含负环)

```
void floyd()
{
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

本质：三维dp：二维数组滚动覆盖

所以 Floyd 是：

按中转点集合逐步扩大的 DP

不是：

按区间长度扩大的 DP。

k 每增加一次，不是在“扩大区间”，而是在“多开放一个中转站”。

2. 它为什么不像普通区间 DP?

普通区间 DP 一般是这种东西：

`</> C++`

```
for (len = 1; len <= n; len++)
    for (l = 1; l + len - 1 <= n; l++) {
        r = l + len - 1;
        for (k = l; k < r; k++)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]);
    }
```

它的核心是：

`dp[l][r]` 表示某个连续区间 `[l, r]` 的最优解。

比如矩阵链乘法、石子合并、最优二叉搜索树，都是在处理一个连续区间。

而Floyd的dp：

1. Floyd 在算什么

定义状态：

$$f[k][i][j]$$

表示：

从 `i` 到 `j` 的最短路长度，
并且这条路径中只允许经过编号 `1~k` 的点作为中转点。

注意这里“中转点”意思是路径中间经过的点，不包括起点 `i` 和终点 `j`。

2. 初始状态为什么对

当 $k = 0$ 时, 表示:

不允许经过任何中转点。

那从 i 到 j 只可能有两种情况:

1. $i == j$, 距离是 0
2. $i \rightarrow j$ 有直接边, 距离就是边权
3. 否则不可达, 设成 INF

这正对应你的初始化:

```
</> C++  
  
if (i == j) d[i][j] = 0;  
else d[i][j] = INF;
```

以及读边时:

```
</> C++  
  
d[a][b] = min(d[a][b], c);
```

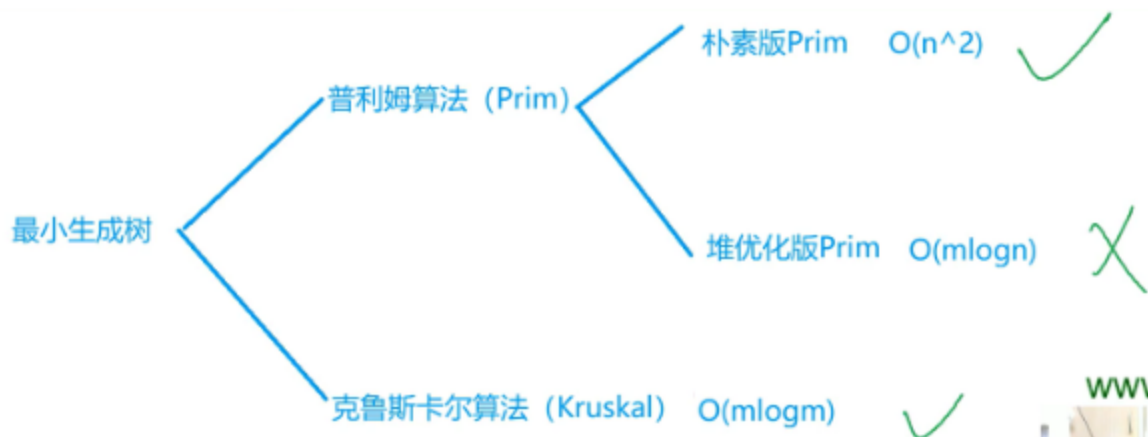
表示重边取最小。

所以初始时, $d[i][j]$ 就是:

$$f[0][i][j]$$

$$f[k][i][j] = \min(f[k-1][i][j], f[k-1][i][k] + f[k-1][k][j])$$

2最小生成树:



2.1 朴素prim (稠密图, $O(n^2)$, 贪心)

① 朴素 Prim

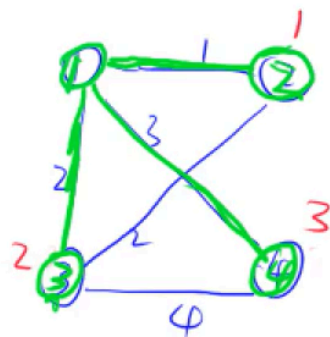
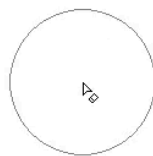
$dist[i] \leftarrow +\infty$

for ($i=0; i < n; i++$)

找集合外距离最近的点

用它更新其他点到集合的距离

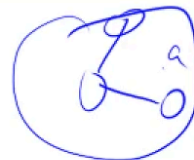
$st[i] = true;$



2.2 Kruskal (稀疏图, $O(m \log m)$ 并查集的应用)

Kruskal 算法

acwing.com - 要退出全屏可以按 Esc

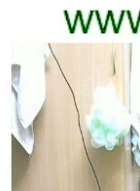


① 将所有边按权重从小到大排序 $O(m \log m)$

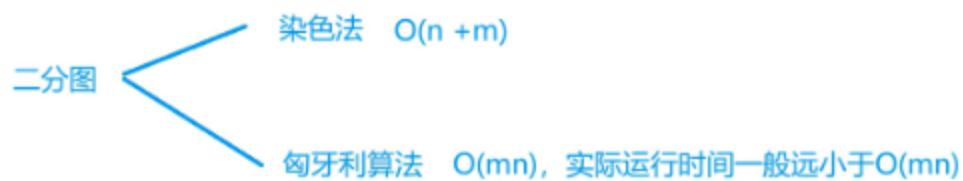
② 枚举每条边 a, b 权重 c

if a, b 不连通

将这条边加入集合中



3二分图

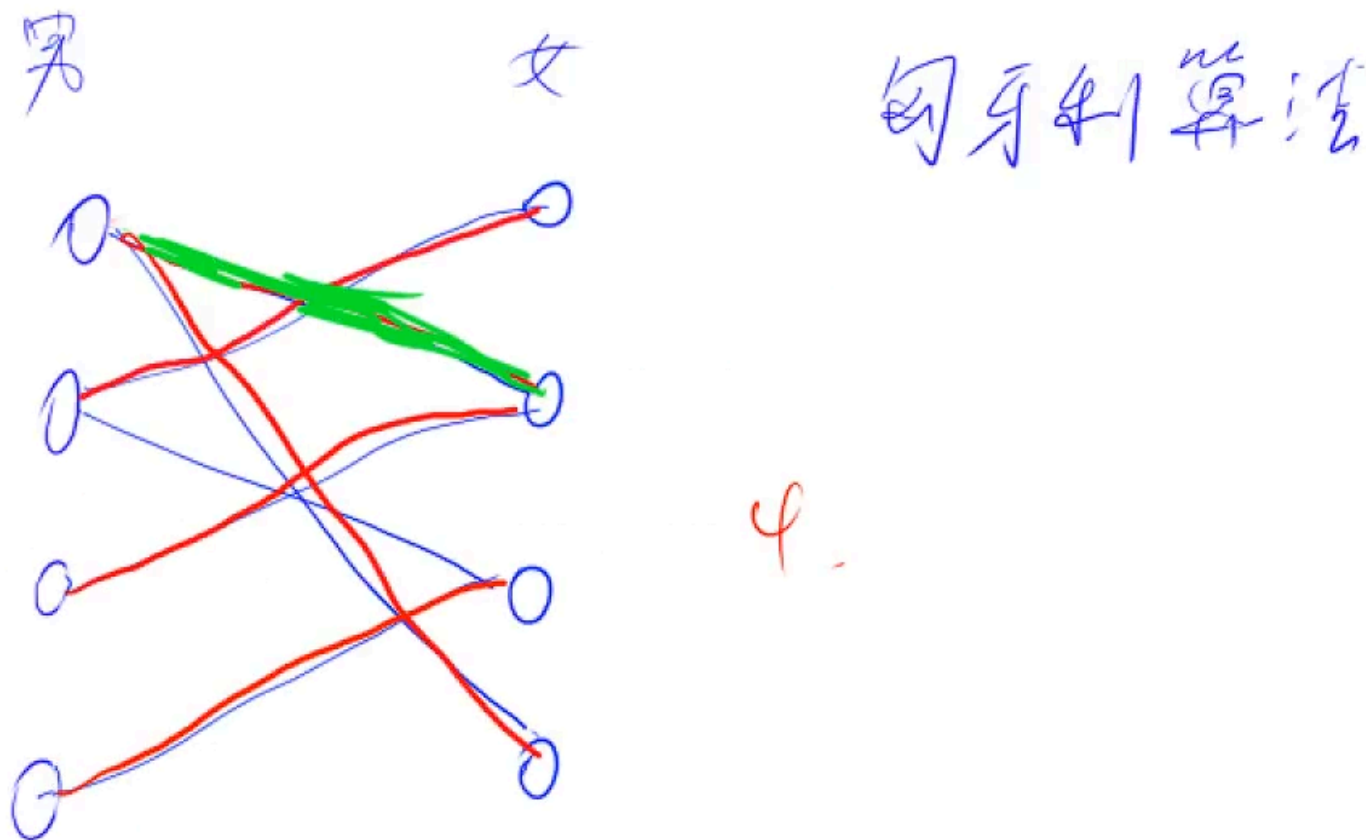


3.1 判断二分图:

二分图当且仅当没有奇数环 $\Leftrightarrow$ ranseguochengzhongshifouch

3.2 二分图最大匹配: 非诚勿扰第一期 (也可建模为通用的最大流)

专门针对二分图最大匹配的高效算法: “匈牙利算法”: 绿帽算法, 让别人去找备胎, 如果有的话, 就把女朋友空出来让给我!



Thought for 7s

你这里说的“匈牙利算法”，其实是 OI/算法题里常说的 **二分图最大匹配的增广路算法**。

它为什么对，核心就一句：

只要还能找到一条增广路，就一定能把匹配数加 1；找不到增广路时，当前匹配就是最大匹配。

这就是它正确性的根。

3. 什么叫增广路

相对于当前匹配 M ，一条路径如果满足：

1. 起点是一个 **未匹配的左点**
2. 终点是一个 **未匹配的右点**
3. 路上的边按
未匹配边、匹配边、未匹配边、匹配边.....
这样交替出现

那么它就叫 **增广路**。

4. 为什么找到增广路就能让匹配数 +1

这是最关键的一步。

沿着增广路，把边状态全部翻转：

- 原来没匹配的边，变成匹配边
- 原来匹配的边，变成非匹配边

4网络流

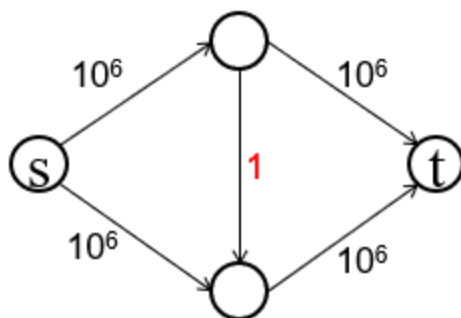
4.1最大流：（非诚勿扰第一期也可以这么做）

4.1.1Fold-Fulkerson（DFS，闭眼睛找，复杂度又数据值决定）

网络流

■ Fold-Fulkerson算法 $c_f(u, v) = \begin{cases} c(u, v) - f(u, v), & \text{若}(u, v) \in E \\ f(v, u), & \text{若}(v, u) \in E \end{cases}$

- 在残留网络从源到汇找一条可行路径（DFS），塞满流量
- 下图，可行路径每次都经过权值为1那条边，时间复杂度很高，复杂度收输入的值决定，不取决于边、点的数量

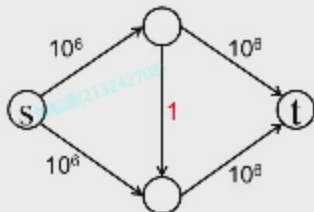


4.1.2Edmonds-Karp（BFS,每次在残留网络中找边数最少的增广路，图结构分析，复杂度由边点数决定）

网络流

■ Edmonds-Karp算法

- 使用BFS搜索可行路径，时间复杂度 $O(VE^2)$



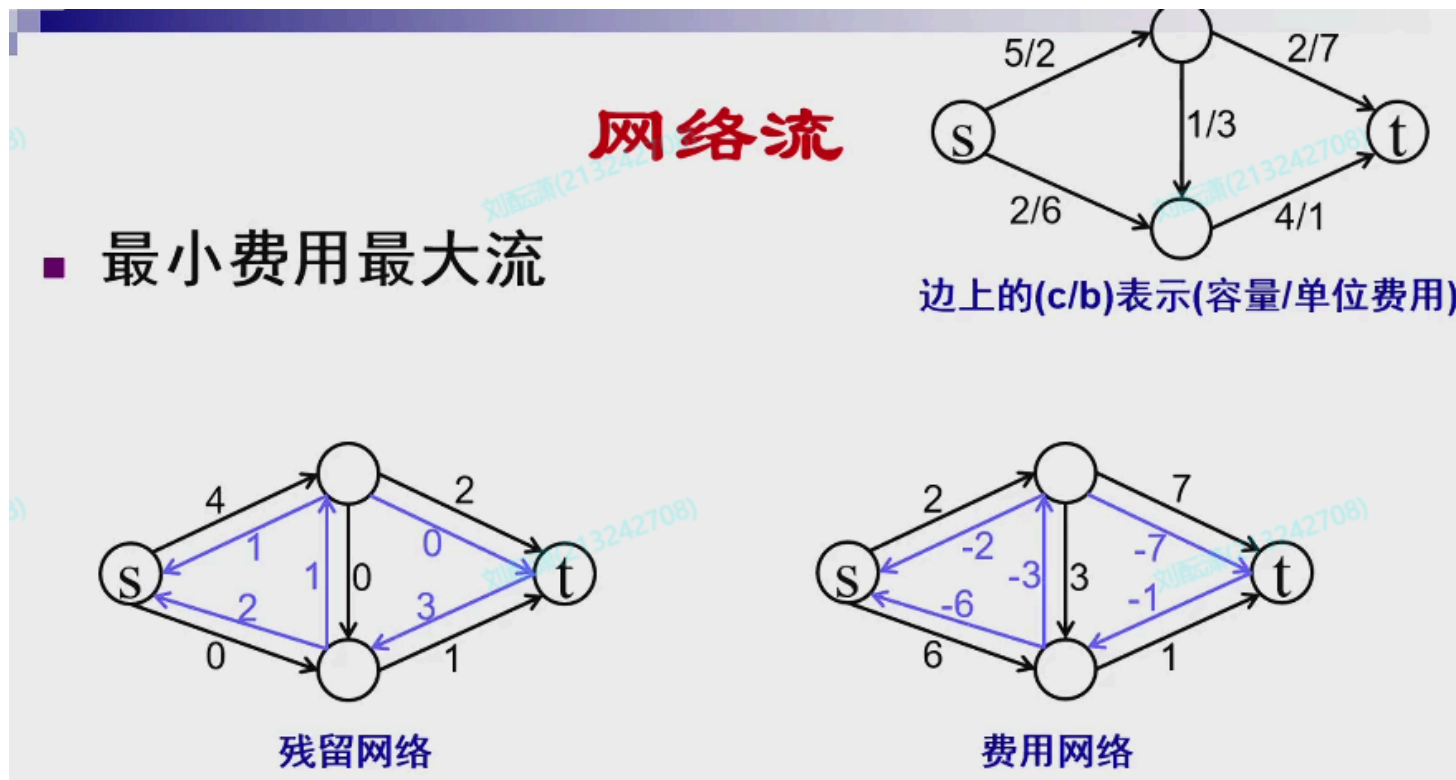
4.1.3 Dinic最大流算法（比朴素的增广路快很多）：

Dinic = BFS 分层 + DFS 找阻塞流

4.2最小费用最大流：非诚勿扰第二期

贪心但是可以反悔（有反向边），不是常规的不可逆贪心，

优先满足的最大流，[流量，费用]



在残留网络中搜索可行的增广路是，不只是像最大流一样DFS\BFS随便找可行路径，而是结合残留网络，每次在费用网络中找最短路径作为增广路

5拓扑排序

queue $\leftarrow$ 所有入度为0的点

一个有向无环图，一定至少存在一个入度为0的点

while queue 不为空

{

$t \leftarrow$ 队头

枚举t的所有出边 $t \rightarrow j$

删掉 $t \rightarrow j$, deg_j--

if $deg_j == 0$;

queue $\leftarrow j$



www.acwing.com



所有入度为0的入队，出队后将相邻节点入度-1

六、STL库

```
1  /*
2
3  vector, 变长数组, 倍增的思想
4  string, 字符串, substr(), c_str()
5  queue, 队列, push(), front(), pop()
6  priority_queue, 优先队列, push(), top(), pop()
7  stack, 栈, push(), top(), pop()
8  deque, 双端队列
9  set, map, multiset, multimap, 基于平衡二叉树（红黑树），动态维护有序序列
10 unordered_set, unordered_map, unordered_multiset, unordered_multimap, 哈希表
11 bitset, 压位
12
13 */
```